ML SYSTEMS

Distributed Training Visual Guide

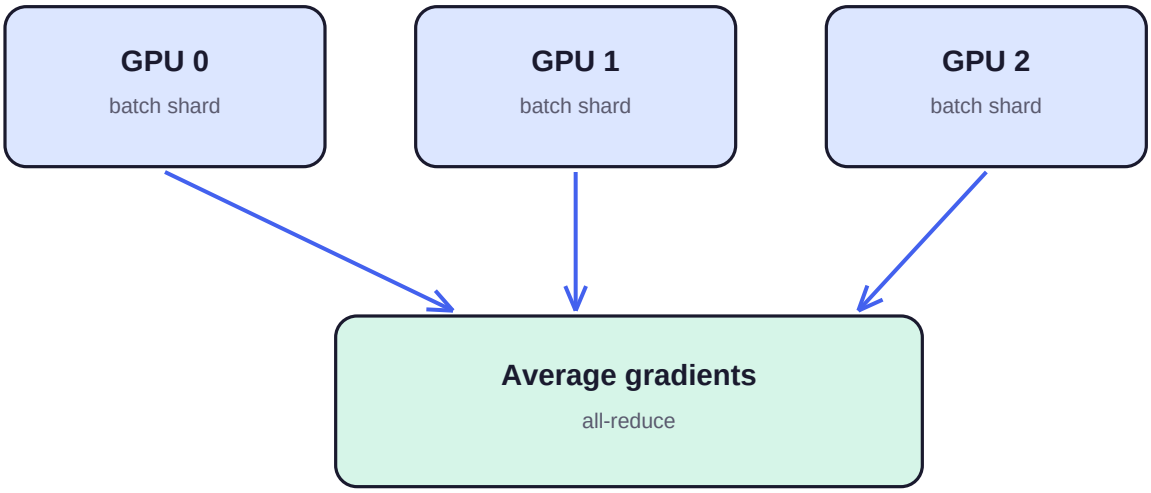
Training across many GPUs:
data parallelism, all-reduce, ZeRO, pipeline parallelism.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/5

Data Parallelism

Same model, different data, on every GPU.



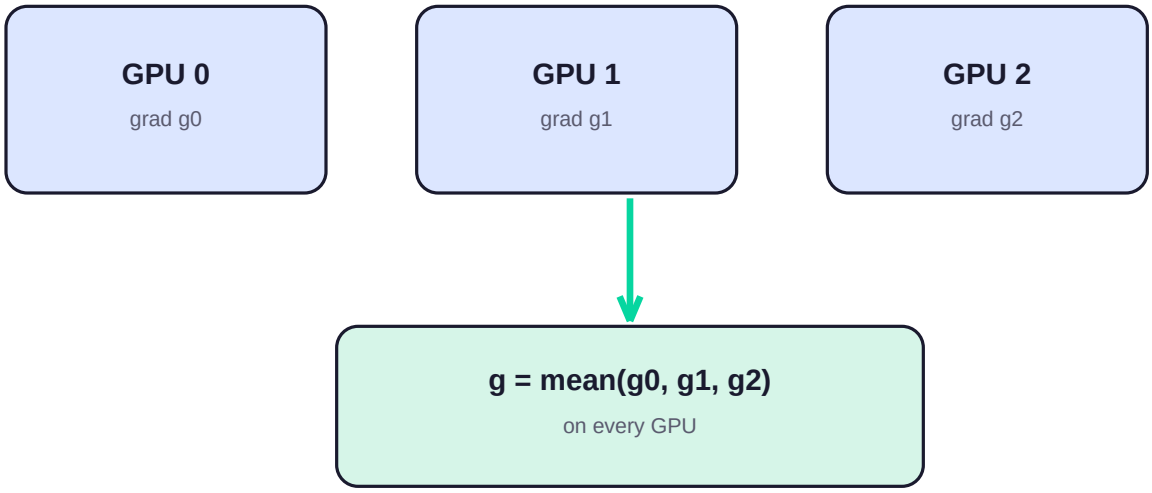
Each GPU computes gradients on its shard; gradients are averaged; every replica applies the same update.

The default strategy. Effective batch size = per-GPU batch x GPU count — scale the learning rate accordingly.

STEP 2/5

All-Reduce

Every GPU ends up with the same averaged gradient.



Ring all-reduce: each GPU exchanges chunks with neighbors — bandwidth-optimal, no central bottleneck.

Communication overlaps with backward-pass computation (gradient bucketing) so GPUs rarely sit idle.

STEP 3/5

Model & Pipeline Parallelism

Split the MODEL when it doesn't fit on one GPU.

Model parallel

layers split across GPUs; activations flow between them.

Pipeline parallel

stages process micro-batches in a pipeline; some idle bubbles.

Pipeline bubbles waste compute — interleaved schedules and more micro-batches shrink them.

STEP 4/5

ZeRO: Sharding the Optimizer

The optimizer states are the real memory hog.

Problem

model size in FP32.

ZeRO-1/2/3

gradients, params, across GPUs.

Result

bigger models.

ZeRO-3 shards everything; each GPU gathers what it needs just in time. The enabler of hundred-billion-param training.

STEP 5/5

Choosing a Strategy

Match the parallelism to the bottleneck.

- Model fits on one GPU? Data parallelism (+ larger batches) is all you need.
- Optimizer memory explodes? Add ZeRO-1/2 — nearly free with data parallelism.
- Model doesn't fit at all? ZeRO-3, or tensor/pipeline parallelism for the biggest models.
- Network slow? Gradient compression and accumulation cut communication.
- Profile first: the bottleneck is usually memory, communication, or data loading — in that order.

STEP ✓/✓

Key Takeaways

Distributed Training — Remember

- Data parallelism: replicate the model, shard the data, average gradients.
- Ring all-reduce synchronizes gradients with no central bottleneck.
- Model/pipeline parallelism splits layers across GPUs when one GPU can't hold the model.
- ZeRO shards optimizer states (and more) to train far bigger models.
- Choose by bottleneck: fit -> data parallel; memory -> ZeRO; huge -> pipeline.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev